

RESEARCH ARTICLE

Dynamic Obstacle Avoidance for an MAV Using Optimization-Based Trajectory Prediction With a Monocular Camera

MIAOJUN ZHOU¹ AND HYEONBEOM LEE², (Member, IEEE)¹School of Electronics Engineering, Kyungpook National University, Daegu 41566, South Korea²Department of Mechanical Engineering, Ajou University, Suwon-si 16499, South Korea

Corresponding author: Hyeonbeom Lee (hbeomlee@ajou.ac.kr)

This work was supported by the National Research Foundation of Korea (NRF) Grant funded by the Korean Government [Ministry of Science and ICT (MSIT)] under Grant RS-2023-00213897 and Grant RS-2024-00411660.

ABSTRACT Vision-based algorithms are widely applied to micro-air vehicles (MAVs) because of their limited takeoff weight. Conventional stereo camera requires a large baseline for long-distance detection, which is difficult for MAVs. The rapidly developing, learning-based, monocular depth estimation method can handle these problems and succeed remarkably well in providing acceptable depth in indoor (maximum distance: 10 m) and outdoor (maximum distance: 80 m) environments. For the safety of an MAV in outdoor environments, we, therefore, propose a monocular-camera-based dynamic avoidance system, along with obstacle motion estimation by depth estimation methods using the Kalman filter. To handle the position uncertainty of a dynamic obstacle and predict its future movement, a polynomial-fitting-based trajectory prediction method with a defined uncertainty range has been used. Subsequently, using quadratic programming (QP), a safe, corridor-based, spatiotemporal trajectory generation method is proposed to ensure the safety of the MAV. We validate the performance of our algorithm through simulation and real-world experiments using an MAV.

INDEX TERMS MAV, depth estimation, trajectory prediction, polynomial optimization.

I. INTRODUCTION

Recently, aerial robots have performed complicated tasks such as aerial photography [1], delivery [2], and surveillance [3] without human intervention. However, vision-based obstacle avoidance for MAVs suffers from some problems. The visual simultaneous localization and mapping (SLAM) [4], [5] offers a sparse depth map for an MAV using a monocular camera, but the existence of dynamic obstacles undermines the detection accuracy. The RGB-D camera-based methods [6], [7], [8] have achieved safe avoidance in indoor dynamic environments, but the short detection range poses a challenge to motion planning and the agility of the mobile robots. Human vision system-like stereo cameras

provide accurate depth estimation based on triangulation [9]; however, they require a relatively large baseline for long-distance detection. The recently developing learning-based monocular depth estimation (MDE) technique provides reasonable depth in indoor environments with a single RGB image input, but the inherent scale ambiguity gives unreliable depth estimation, which causes difficulties in safe outdoor navigation. Our study focuses on solving the aforementioned problems for MAVs by designing an integrated framework to handle the dynamic obstacles in an outdoor environment. For the outdoor scenarios, we assumed that the aerial robot is flying in a forest [10] or subterranean space [3]; thus, the obstacles exist on a horizontal plane.

- We propose a position estimation method for dynamic obstacles using learning-based MDE and a monocular camera. The estimation accuracy of our method is

The associate editor coordinating the review of this manuscript and approving it for publication was P. Venkata Krishna^{1b}.



FIGURE 1. Snapshot showing the tiny quadrotor avoiding a pedestrian in an outdoor environment. **Top right:** RGB image taken by an onboard camera. **Bottom right:** Depth image generated by learning-based MDE.

improved by solving the scale-ambiguity problems using object detection and a height-based depth estimation algorithm.

- We have proposed a safe-corridor-based, spatiotemporal motion planning method to generate a safe and efficient spatiotemporal trajectory. Unlike the trajectory prediction that does not consider uncertainty [6], our algorithm with uncertainty estimation can handle the uncertain movement of obstacles. In addition, unlike in uncertainty prediction using a simple constant velocity model [8], we can effectively respond to the changes in the speed of dynamic obstacles by predicting their trajectory based on past movement and predicting their movement uncertainty accordingly.
- Finally, our paper provides an integrated framework for avoiding dynamic obstacles using a monocular camera. Thanks to our framework, our approach enables more accurate prediction of the trajectory of moving obstacles using a monocular camera, resulting in faster navigation to the target point when compared to existing algorithms. We validate our proposed method in simulations and real-world outdoor environments using a tiny quadrotor equipped only with a monocular camera. Our video including simulations and experiments can be shown on YouTube.¹

II. RELATED WORKS

Recent studies on vision-based dynamic obstacle avoidance [6], [7], [8] generate an optimization-based trajectory based on a specific map (ESDF, point cloud or voxel map) with the use of an RGB-D camera. RGB-D cameras have a limited detection range and are susceptible to sunlight in outdoor environments; therefore, these methods [6], [7], [8] may be inadequate for safe outdoor navigation. In addition, to handle dynamic obstacles in a simple way, they predict the trajectory of an obstacle with a constant velocity model, which is incompatible with long-range and complex-behavior obstacle prediction. Another popular approach is the model

predictive control (MPC)-based method [11], [12]. An avoidance method with a dynamic-obstacle-avoidance risk region is proposed in [11]. This paper was verified for dynamic obstacles in an outdoor environment. However, it also uses a simple, constant velocity model at different sampling times, so responses to obstacles with changing speeds may not be effective. To address this issue, research was carried out on dynamic avoidance with the time dimension studied using the spatiotemporal [13], [14] information of a dynamic obstacle. They showed the reachable set of dynamic obstacles with the time dimension introduced but suffered from the time irreversibility problem during the planning and could not satisfy the kinematic constraints. In our work, this problem is resolved using safe corridors and optimal planning. In addition, unlike [13], [14], we validate our approach through experiment, not simulation.

Deep-learning-based obstacle avoidance has been realized using a monocular camera [15], [16]. In [15], the authors designed a residual convolutional architecture that produced an avoidance command for each single input image without using any complex algorithms. However, this method cannot handle dynamic obstacles with complex behavior because it cannot predict obstacle trajectories. After Eigen et al. [17] first employed a CNN for monocular depth estimation, accurate MDE algorithms were developed in several studies [18], [19], [20]. In [16], the MDE algorithm was first employed in a behavior arbitration scheme to navigate a quadrotor for avoiding obstacles. However, this method was only validated in indoor environments with static obstacles; while our work exhibited a robust and wide detection range in outdoor environments and enabled safe navigation with dynamic obstacles.

III. UNCERTAINTY-AWARE ENVIRONMENT PERCEPTION

A. DEPTH ESTIMATION

Depth images are widely used for the environmental perception. Since Eigen et al. [17] first used a CNN for monocular depth estimation, huge amounts of data and computational resources have been poured into the development of deep learning-based MDE algorithms [18], [19], [20]. Among these state-of-the-art works, we employ Adabins [20], which produces a high-quality, dense depth map by using a combination of CNNs and Transformer networks with a proposed adaptive, bin-width estimator block. In the case of Adabins, to obtain the depth image, the process takes only 90 milliseconds in laptop computer equipped with an RTX 3050 Mobile, an Intel Core™ i7-11800H 2.3GHz processor, and 16 GB of RAM. Nevertheless, learning-based depth estimation alone is inherently scale-ambiguous or inaccurate when applied in an environment that differs from the training data. Therefore, we propose a depth correction method based on a simple but effective height-based depth estimation method [21] under the assumption that the rough height H of the dynamic obstacle is known. We consider humans or vehicles, which are the most common dynamic

¹https://youtu.be/_WEEef1EgaQ

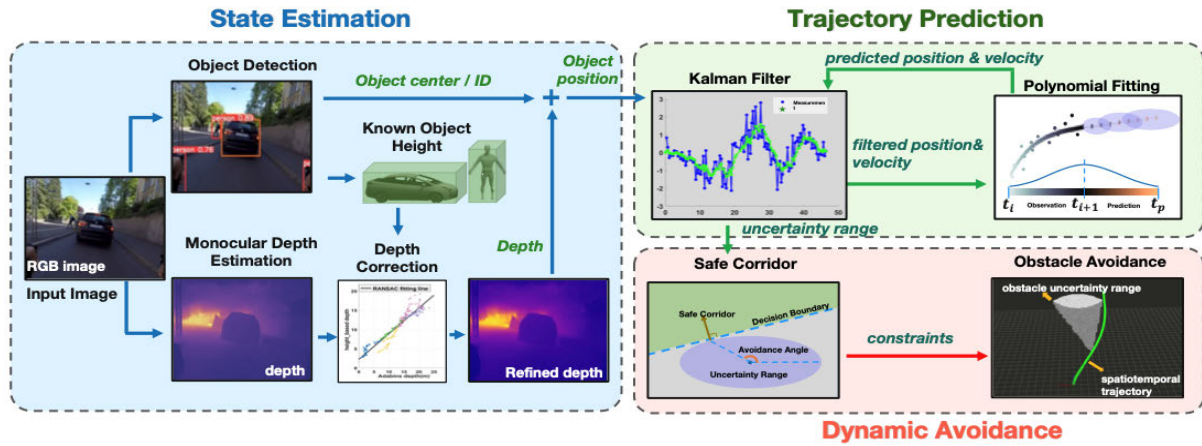


FIGURE 2. Monocular-camera-based dynamic avoidance system diagram.

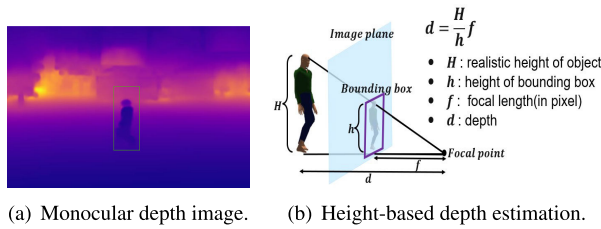


FIGURE 3. Depth estimation.

objects in the outdoors as obstacles. The bounding box generated by YOLO-v5 [22] serves as both an object detector and a depth reference. As shown in Fig.3(b), the camera model is applied to derive the depth d as:

$$d = \frac{H}{h} * f \quad (1)$$

where f is the focal length, and h is the height of the bounding box. Note that the error in the height-based depth has an error as follows:

$$h = \frac{H_t}{d_t} f, \quad d_w = \frac{H_w}{H_t f / d_t} f = \frac{H_w}{H_t} d_t, \quad (2)$$

where H_t is the actual height of an object, H_w is the wrong height, d_t is the true depth, and d_w is the estimated depth using H_w . Therefore the depth error with the height estimator is $|d_t - d_w| = |(H_t - H_w)d_w/H_w|$. For example, with $H_t = 1.7\text{m}$ $H_w = 1.5$ or $H_w = 1.9$ (about 20 cm error), the depth error would be about 2m when the object is 15m away (a maximum error of 13.3 %). In the case of a 5 m obstacle, the height-based method estimates 4.34 m depth(i.e., 0.65 cm error), which is sufficient for obstacle avoidance and is smaller than that estimated by MDE and shows a better depth estimation consistency than MDE. For this reason, we fused height estimation with MDE.

The results of the proposed depth correction method are shown in Fig.4. Inspired by [23], the estimated

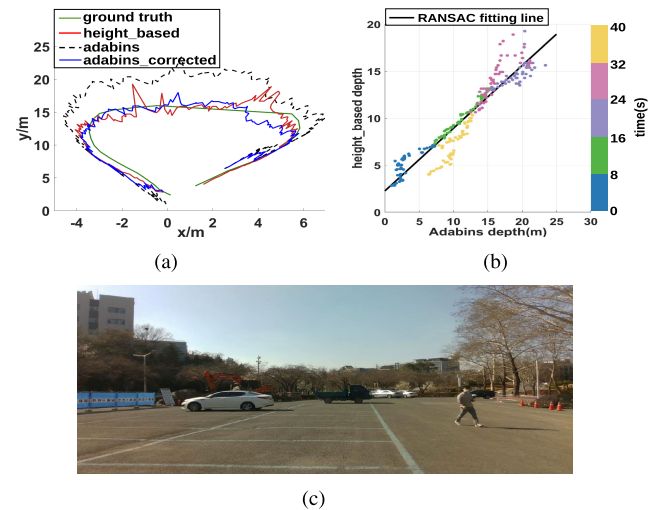


FIGURE 4. Depth correction. (a): Comparison of depth data, (b): Linear relationship between the Adabins depth and height-based depth detection, (c): Position collection of a pedestrian using RTK-GPS.

depth can be corrected by the linear regression analysis as

$$\mathbf{d} = \alpha \times \hat{\mathbf{d}} + \beta, \quad (3)$$

where α and β are the regression values. $\hat{\mathbf{d}}$ is the estimated depth obtained by MDE and $\mathbf{d}$ is the true depth. Our main objective is to compute α and β using the estimated depth by Adabins and height-based depth(See Fig. 4(b)). The correction is performed because MDE may suffer from scale ambiguity. We used RANSAC between the Adabins depth and the height-based depth and generated a fitting line to obtain α and β .

To validate this effect, we record the movement of a person using a camera and compare the depth estimation results. To obtain the ground-truth position of a person in outdoors, we employed EMLID Reach RS2 RTK-GPS, which returns the distance between the GPS base and the rover

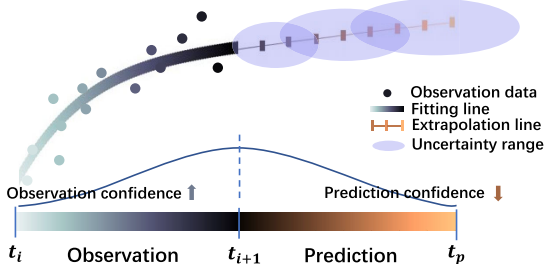


FIGURE 5. Uncertainty-aware trajectory prediction.

with centimeter precision. The ground-truth trajectory of a person holding the rover is depicted in green in 4(a), and the results based on height-based depth estimation and Adabins are illustrated as red and black lines respectively. To solve the scale-ambiguity of Adabins, we use the linear relationship between the Adabins depth and the height-based depth and generate a fitting line referring to RANSAC (see Fig.4(b)). The Adabins depth is corrected based on this fitting line, and the corrected result is drawn as the blue line in Fig.4(a). Based on our proposed depth refinement method, the detection range stretches to nearly 15 m with robustness guaranteed in various environments.

B. DATA FUSION

We apply the Kalman filter to improve the accuracy of the estimation during the fusion of the dynamic model and depth measurement. The state of the obstacle is defined as $\mathbf{x} = [x, \dot{x}, y, \dot{y}]^T$, representing the position and velocity in x and y directions, respectively. The dynamic system can be expressed as:

$$\begin{aligned} \mathbf{x}_{k+1} &= F\mathbf{x}_k + N(0, Q) \\ z_k &= H\mathbf{x}_k + N(0, R) \end{aligned} \quad (4)$$

where F is a state transition matrix for the dynamic model, and H is a measurement matrix for the depth data. Q and R define the process noise and measurement noise, both of which are assumed to follow zero-mean Gaussian distributions.

C. UNCERTAINTY-AWARE TRAJECTORY PREDICTION

Predicting a dynamic obstacle trajectory in a short time-horizon is crucial for autonomous vehicle navigation in a dynamic environment. In [24], the trajectory prediction problem was addressed by fitting a polynomial, but the uncertainty of motion was not taken into account. To increase the safety in outdoor environments, we propose an uncertainty-aware trajectory prediction method. Based on the frame in [24], we have added a velocity-loss term to make the prediction model more robust to abrupt velocity changes. As depicted in Fig.5, the optimal trajectory is divided into two sections. The first section pertains to curve fitting during the observation interval $[t_i, t_{i+1}]$. The second section pertains to extrapolation during the prediction interval $(t_{i+1}, t_p]$. The

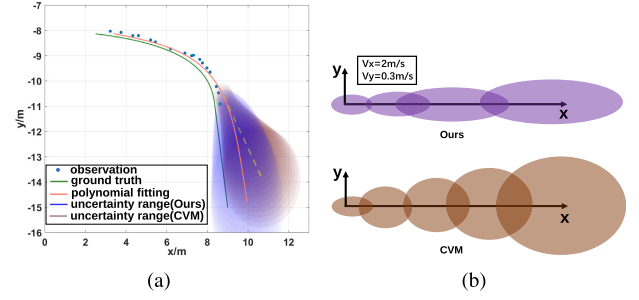


FIGURE 6. Comparison of occupancy efficiency.

trajectory of the dynamic obstacle is assumed to be a Bezier curve:

$$\hat{P}_i(t) = \sum_{j=0}^n c_{i,j} b_n^j \left(\frac{t-t_i}{t_p-t_i} \right) \quad t \in [t_i, t_{i+1}] \quad (5)$$

where $c_{i,j}$ is the j th control point of the Bezier curve, $b_n^j(\cdot)$ denotes the Bernstein based on polynomial function, n denotes the polynomial degree, and t_p is the terminal of the prediction interval. The core idea of polynomial fitting in this study is to find the best polynomial function that matches a sequence of observed position and velocity data while satisfying dynamical constraints. Thus, the trajectory prediction can be boiled down to an optimization problem defined as:

$$\begin{aligned} \min_{\hat{P}_i} \quad & \sum_{k=0}^m w_k (\|\hat{P}_i(t_k) - p_k\|_2^2 + \|\hat{P}_i(t_k) - v_k\|_2^2) + \lambda \int_{t_i}^{t_p} \|\ddot{\hat{P}}_i(t)\|_2^2 dt \\ \text{s.t.} \quad & -v_{\max} \leq \dot{\hat{P}}_i(t) \leq v_{\max} \\ & -a_{\max} \leq \ddot{\hat{P}}_i(t) \leq a_{\max} \end{aligned} \quad (6)$$

where the loss function, designated as L_2 , is the difference between the estimated and observed data. We add a velocity-loss term, making the prediction more sensitive to sharp turns. An acceleration-regulator term was added to attenuate overfitting. p_k and v_k define the position and velocity data estimated by the Kalman filter at time t_k , and m is the number of observed data points during the interval $[t_i, t_{i+1}]$. w_k ($w_k > 0$) is the observation weight, and it increases over time as a recently observed data point weighs more than a previous data point when deciding the trajectory of future movement. λ defines the regulator weight. $v_{\max}$ and $a_{\max}$ define the maximum velocity and acceleration, respectively.

This optimization problem can be reformulated into a QP problem by first rewriting the trajectory function in matrix form:

$$\hat{P}_i(t) = \sum_{j=0}^n c_{i,j} b_n^j \left(\frac{t-t_i}{t_p-t_i} \right) = \sum_{j=0}^n p_{i,j} \left(\frac{t-t_i}{t_p-t_i} \right)^j = T_i^T(t) P_i$$

$$T_i(t) = \begin{bmatrix} \left(\frac{t-t_i}{t_p-t_i}\right)^0 \\ \vdots \\ \left(\frac{t-t_i}{t_p-t_i}\right)^n \end{bmatrix}, \mathbf{P}_i = \begin{bmatrix} p_{i,0} \\ p_{i,1} \\ \vdots \\ p_{i,n} \end{bmatrix} = M \cdot \begin{bmatrix} c_{i,0} \\ c_{i,1} \\ \vdots \\ c_{i,n} \end{bmatrix} = M \cdot \mathbf{C}_i \quad (7)$$

where M is a mapping matrix transforming the Bezier curve control point matrix $\mathbf{C}_i$ to the standard polynomial coefficient matrix $\mathbf{P}_i$. In order to convert (6) into quadratic optimization problem using matrix form, the position loss term L_2 can be reformulated as:

$$\begin{aligned} & \sum_{k=0}^m w_k \left(\|\hat{\mathbf{P}}_i(t_k) - p_k\|_2^2 \right) \\ &= \left\| \underbrace{\begin{bmatrix} \sqrt{w_1} & & \\ & \ddots & \\ & & \sqrt{w_m} \end{bmatrix}}_W \left(\underbrace{\begin{bmatrix} T_i^T(t_0) \\ \vdots \\ T_i^T(t_m) \end{bmatrix}}_{T_i} \mathbf{P}_i - \underbrace{\begin{bmatrix} p_1 \\ \vdots \\ p_m \end{bmatrix}}_p \right) \right\|_2^2 \\ &= \mathbf{P}_i^T (T_i^T W^T W T_i) \mathbf{P}_i - 2 (T_i^T W^T W p)^T \mathbf{P}_i + p^T W^T W p \\ &= \mathbf{C}_i^T (M^T T_i^T W^T W T_i M) \mathbf{C}_i - 2 (M^T T_i^T W^T W p)^T \mathbf{C}_i + \delta_p \\ &:= \mathbf{C}_i^T H_p \mathbf{C}_i - q_p^T \mathbf{C}_i + \delta_p \quad (8) \end{aligned}$$

Similarly, the velocity-loss term can also be written as:

$$\sum_{k=0}^m w_k \left(\|\hat{\mathbf{P}}_i(t_k) - v_k\|_2^2 \right) = \mathbf{C}_i^T H_v \mathbf{C}_i - q_v^T \mathbf{C}_i + \delta_v \quad (9)$$

By referring to [10], the regulator term can be written in quadratic form as:

$$\lambda \int_{t_i}^{t_p} \|\ddot{\hat{\mathbf{P}}}_i(t)\|_2^2 dt = \lambda \mathbf{P}_i^T Q \mathbf{P}_i, \quad (10)$$

where Q is a hessian matrix. The detailed derivation of (8) and (9) can be found in [10]. By combining (7)-(10), the optimization problem is summarized as a standard quadratic optimization:

$$\min_{\mathbf{C}_i} \mathbf{C}_i^T H \mathbf{C}_i + q^T \mathbf{C}_i + \delta \quad (11)$$

$$\text{s.t. } A_{ieq} \mathbf{C}_i \leq b_{ieq} \quad (12)$$

where $\mathbf{C}_i$ is the variable to be optimized. The terms A_{ieq} and b_{ieq} represents the dynamical constraints using (6) and (7). H , q , and δ can be computed as follows:

$$H = H_p + H_v + \lambda Q \quad (13)$$

$$q = q_p + q_v, \delta = \delta_p + \delta_v \quad (14)$$

To fully account for the motion uncertainty of dynamic obstacles, the predicted positions are taken as measured values and substituted back into the Kalman filter to generate the uncertainty range. As the uncertainty range is coupled with time and velocity, the prediction inaccuracy increases over time, with a larger velocity corresponding to a wider

accessible range. We set the measurement noise matrix as a time-variant function:

$$R(t) = kv(t)t^2 R_0 \quad t \in (t_{i+1}, t_p] \quad (15)$$

$$v(t) = \begin{bmatrix} \hat{P}_x(t) & 0 \\ 0 & \hat{P}_y(t) \end{bmatrix} \quad (16)$$

where $v(t)$ ($v(t) \in \mathbb{R}^{2 \times 2}$) is a diagonal matrix with velocity in the x and y directions at time t as diagonal entries. R_0 ($R_0 \in \mathbb{R}^{2 \times 2}$) is the initial measurement noise matrix. k is a user-defined scaling parameter. The diagonal entries of the covariance matrix outputted by the Kalman filter define the variance of each state variable. Based on this, we generate covariance ellipses in Fig.5 to illustrate the uncertainty range propagation. The uncertainty range is conducive to subsequent collision-free trajectory generation. Fig.6 shows a qualitative comparison with the constant velocity model (CVM), where the trajectory prediction is based on a constant, estimated velocity [7], [8], [11]. Fig. 6(a) shows that the proposed method predicts the obstacle's future movement better. In fig.6(b), the velocity in the x direction is much larger than the velocity in the y direction, and this velocity difference is reflected in the uncertainty range expanding rate in our method, which shows superior efficiency in establishing the configuration space.

IV. RISK-AWARE SPATIO-TEMPORAL (RAST) TRAJECTORY GENERATION

This section focuses on the spatiotemporal trajectory generation method. The crux of this method lies in projecting the time-stamped occupancy of the dynamic obstacle in the time dimension and finding a collision-free trajectory in this spatiotemporal configuration space. In comparison with space-only methods that update the solution at a high frequency or project all the possible occupancies in one planning frame and achieve an over-conservative path, the proposed method alleviates the computational burden caused by real-time updates and incomplete solutions in dense environments. However, several difficulties make the traditional path planners, such as the A* or RRT* algorithms, inapplicable to such a problem. 1). The irreversibility of time: The spatiotemporal trajectory cannot swing in the time-increasing direction. 2). Kinematic constraints: As the time dimension is involved, the geometric features of the spatiotemporal trajectory connect with the kinematic property; for instance, the gradients in the x and y directions are bound with the velocity. 3). The optimal solution: The irregular, cone-shaped, spatiotemporal obstacle makes it hard to find an optimal trajectory with guaranteed feasibility and efficiency. The previous work [13], [14] assumes that the robots' route is fixed on a predefined line and degrades the original problem to a 2D path planning problem. However, the suboptimal 2D solution is inappropriate for complex tasks. To overcome these challenges, we extend a conventional safe-corridor-based method to generate a viable spatiotemporal trajectory.

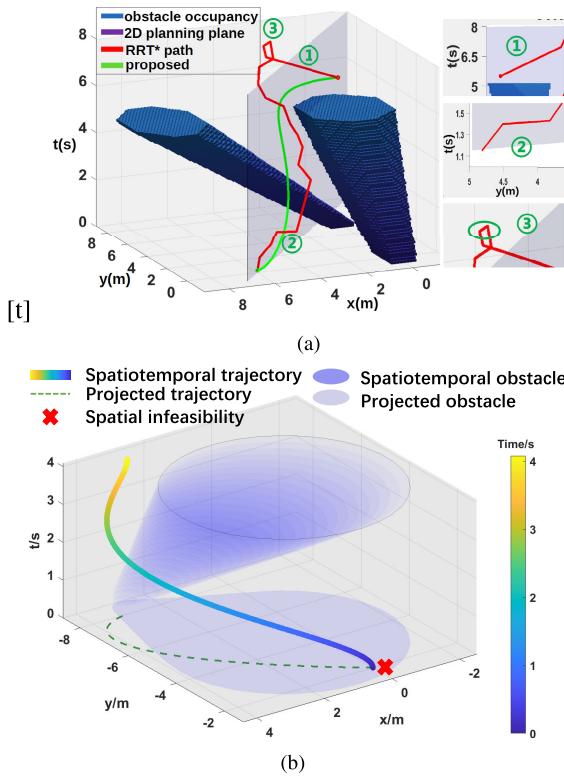


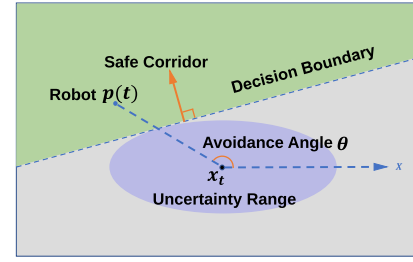
FIGURE 7. Spatiotemporal trajectory generation: (a).RRT* path, ①: path went from 6.7 s to 6.5 s, violating the time irreversibility principle. ②: path transversed 1.2 m in the y direction within 0.4 s, violating the velocity constraints. ③: sharp turns. (b): proposed safe-corridor-based spatiotemporal trajectory, the red cross indicates that the problem is unsolvable when only spatial occupancy is considered.

A. SAFE CORRIDOR GENERATION FOR RAST

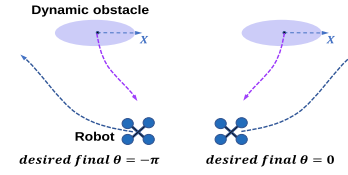
The constraints for the trajectory to be outside the elliptical uncertainty range can be formulated as:

$$(x_t - p(t))^T Q_t^{-1} (x_t - p(t)) > 1 \quad (17)$$

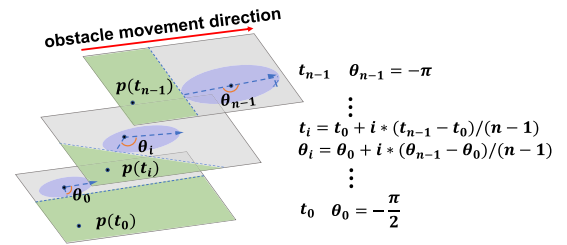
where $p(t) \in \mathbb{R}^2$ is the position of the robot at time t in a horizontal plane, $x_t \in \mathbb{R}^2$, and $Q_t \in \mathbb{R}^{2 \times 2}$ defines the mean and the covariance matrix of the ellipse at the same time. The positive definite matrix Q_t^{-1} makes the original problem nonconvex, which is hard to solve. To relax the problem to a convex form, we take the tangent of the ellipse as a decision boundary. The side with no obstacle is defined as the safe corridor. To explain the avoidance strategy, we define the angle between the X-axis of the ellipse and the line connecting the ellipse center and the desired robot position as the avoidance angle θ (see Fig.8(a)). To make the avoidance direction the inverse of the obstacle's direction of motion, we set the final avoidance angle as 0 when the obstacle moves to the left based on the initial position of the robot, and set it to $-\pi$ when the obstacle moves to the right. For example, in Fig.8(c), the initial avoidance angle θ_0 at t_0 is $-\frac{\pi}{2}$; as the obstacle moves toward the positive direction of X-axis, the robot will divert to the minus direction. Thus, the final avoidance angle θ_{n-1} at t_{n-1} is set as $-\pi$. To mitigate the



(a) Safe corridor decision boundary.



(b) Avoidance angle change.



(c) Avoidance angle change.

FIGURE 8. Layer-formed safe corridor generation.

violation of dynamic constraints caused by a drastic shift of the safe corridor, we divide the avoidance angle evenly by a user-defined layer number n ; then the avoidance angle at t_i is $\theta_0 + i * (\theta_{n-1} - \theta_0) / (n - 1)$ ($i = 0, 1, \dots, n - 1$). A larger layer number smoothens the safe corridor change but increases the computational overhead, while a smaller layer number does the opposite. After the generation of the safe corridor, the desired trajectory is optimized. Detailed information can be found in [10].

B. TRAJECTORY OPTIMIZATION FOR STATIC OBSTACLES

Following the description of [10], by fully employing the convex hull property of the Bernstein based on polynomial function, which confines the whole trajectory within the convex hull formed by the control points, we restrain the trajectory within a safe corridor with linear inequalities.

$$\min_P \int_{t_{i+1}}^{t_p} \|\ddot{P}(t)\|_2^2 dt + \eta \|P(t_p) - P_g\|_2^2 \quad (18)$$

$$\text{s.t. } A'_{eq} C' = b'_{eq} \quad (19)$$

$$A'_{ieq} C' \leq b'_{ieq} \quad (20)$$

where $P(t)$ is the planned trajectory, P_g defines the desired goal position, and η is a user-defined weight. The cost function is designed as a total jerk of the whole trajectory to guarantee smoothness. The term $\eta \|P(t_p) - P_g\|_2^2$ represents a goal position loss. The equality constraint (19) counts for the kinematic constraints and continuity in the joints, and the

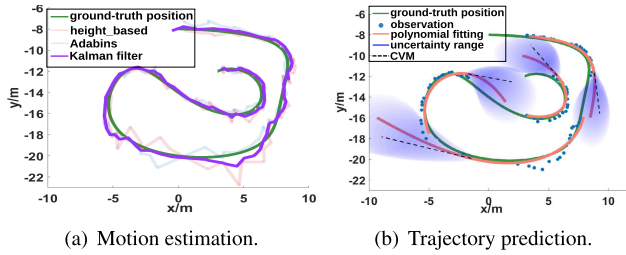


FIGURE 9. Dynamic environment perception.

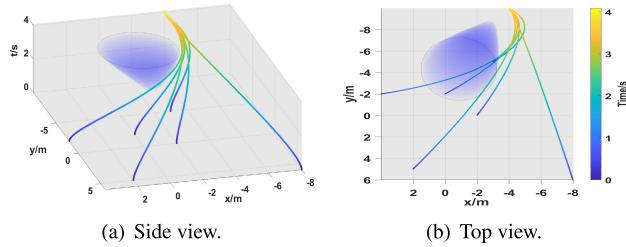


FIGURE 10. Multi-spatiotemporal trajectory generation.

inequality constraint (20) counts for the dynamic feasibility and safe guarantee. Similar to the trajectory prediction section, this problem can also be solved using a QP solver.

V. SIMULATION RESULTS

The environment for simulation is based on the Gazebo simulator with a laptop computer running Ubuntu 18.04 with an RTX 3050 Mobile, an Intel Core™ i7-11800H 2.3GHz processor, and 16 GB of RAM. We used the RotorS simulator [25] to create a drone model loaded with a camera. Adabins, used for depth image estimation, requires 90 ms on a laptop computer. Additionally, the total computation time to calculate the desired trajectory, which includes risk-aware spatio-temporal trajectory, is approximately 134 ms on a desktop computer. Therefore, the overall computation time is around 224 ms.

A. DYNAMIC ENVIRONMENT PERCEPTION

By using the Gazebo simulator, we created an outdoor simulation environment (see Fig. 11(a)) with pedestrians walking in a random smooth route, while a drone loaded with a monocular camera hovered above the origin and estimated one of the pedestrian's coordinates. Fig. 9(a) shows the result of the motion estimation. The green line represents the ground-truth position of the human, the light red line and the blue line depict the results based on the height-based and learning-based MDE, respectively. The purple line shows the final estimated results obtained using the Kalman filter. Fig. 9(b) further shows the result of the trajectory prediction. We take samples of observation data in different situations (depicted as blue dots). The orange lines show the results of the polynomial fitting. The expanding purple ellipses represent the predicted uncertainty range in the next 4 s. We also compared it with the CVM, which is visualized as

TABLE 1. Performance comparison for the efficiency of trajectory prediction.

	CVM	Ours
Prediction Error	1.61	1.19
URE	2.09	4.85

black dotted lines. The metric for prediction performance is defined as the average error between prediction and ground truth:

$$\text{Prediction Error} = \frac{1}{n} \sum_{i=1}^n (\|P_{\text{pred}}(t_i) - P_{\text{GT}}(t_i)\|) \quad (21)$$

where n is the total number of data samples for performance validation, and $P_{\text{pred}}(t_i)$ and $P_{\text{GT}}(t_i)$ define the predicted position and ground-truth at the same time t_i . We also define a metric for the uncertainty range efficiency (URE) as the ratio of the square of the length covered in the predicted uncertainty range to the occupancy volume:

$$\text{URE} = \frac{\text{length of covered trajectory}^2}{\text{occupancy volume}} \quad (22)$$

The comparison results shown in the table illustrate the superiority of our method.

B. SPATIOTEMPORAL TRAJECTORY GENERATION

To verify the robustness of our method, we generated multiple 3D spatiotemporal trajectories by changing only the position of the initial point and relaxing the dynamic constraints. Multiple trajectories with various start points and velocities converge on the destination. Fig. 11(a-b) shows multiple spatiotemporal trajectories with goal point $[-4, -10, 4]$ and start points $[-8, 6, 0]$, $[-2, 0, 0]$, $[0, -2, 0]$, $[4, -2, 0]$, $[2, 5, 0]$, $[-3, -5, 0]$.

C. DYNAMIC AVOIDANCE

We further validated the dynamic avoidance and performed a comparison experiment with another monocular camera-based obstacle avoidance algorithm [15] and an optimization-based algorithm [26]. The simulation results are shown in Fig. 11(a-b). Two dummies were designated to walk toward the quadrotor, the global path from start to end was assumed to be given, and the motion of the dummies was unknown. The quadrotor followed the global path and switched to a local path when dynamic obstacles were detected. Fig. 11(c) shows the comparison result between our work and that of the dronet [15]. Dronet is a learning-based, obstacle avoidance model that takes in a monocular camera image and outputs related collision probability and steering angle. Since there is no prediction of the obstacle's future movement in this method, as shown by the orange line, the steering instructions lead the quadrotor to the obstacle and cause a collision. The brown line shows the situation when the gain for velocity adjustment is increased; the drone reduces speed when encountering an obstacle and steers away. But this stop-and-go strategy wastes a lot of time.

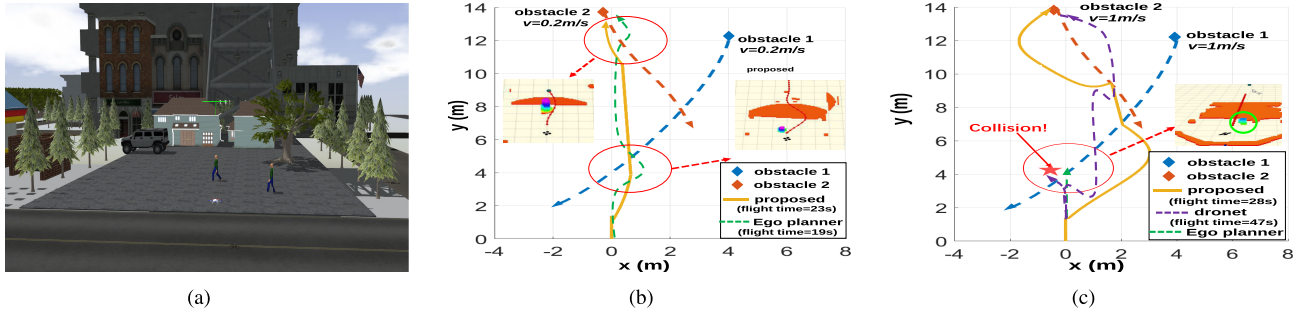


FIGURE 11. Dynamic avoidance: (a) gazebo simulation with dynamic obstacles, (b) Avoidance for slow dynamic obstacles, and (c) Avoidance for fast dynamic obstacles.

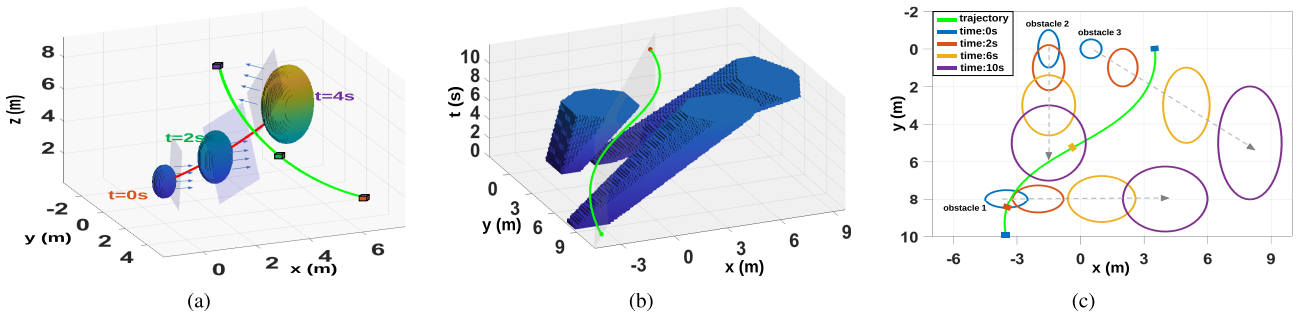


FIGURE 12. Obstacle avoidance for various scenarios: (a) 3D obstacle avoidance, (b) Spatio-temporal trajectory for multiple obstacles and (c) 2D Trajectory for multiple obstacles.

The trajectory generated by our method is depicted by the green line. The average flight time of [15] is 47 s and ours is 28 s. This shows that our method outperforms [15] in efficiency. Ego-planner is an optimization-based algorithm. For slow-moving obstacles, both the Ego-planner and our proposed paper exhibited good performances (Fig. 11(b)). However, when the speeds of the dynamic obstacle increased, the Ego-planner and Dronet collided with the obstacle, but the algorithm proposed in this paper effectively avoided the obstacles (Fig.11(c)). The crashing of the Ego planner is presumed to be due to ineffective recognition of dynamic obstacles.

In Fig. 12, the spatio-temporal trajectories in the presence of 3D obstacle and three 2D obstacles are shown. From the simulation results, we verified that our algorithm can be applied to 3D or multiple obstacles while estimating their uncertainty ranges.

D. EXPERIMENT SETUP

For real-world validation, we used a compact-size quadrotor, DJI Tello, which weighs only 80g and is equipped with a camera providing a 1280×720 pixels RGB image at 30 Hz and a simple optical flow-based odometry system. Communication was achieved using a wireless, 2.4 GHz Wi-Fi connection based on the Robot Operating System. All the computations were performed in a laptop with Intel CoreTM i7-10750H 2.6 GHz and 16 GB RAM. The experimental scenario was set up on a lawn with a few

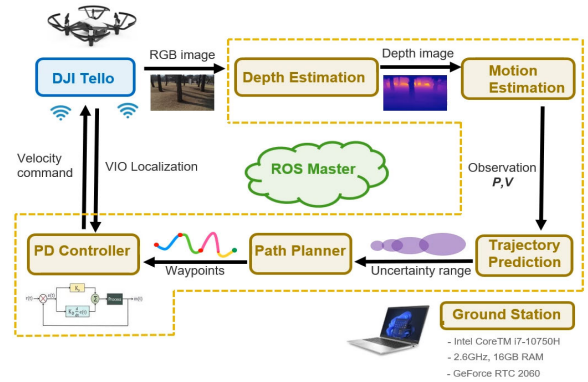


FIGURE 13. Experimental setup.

trees as static obstacles. Two pedestrians serving as dynamic obstacles would start to walk toward the quadrotor from about 10 m away at approximately 1 m/s. The map was assumed to be given, and we generated a minimum snap trajectory referring to [10]. The quadrotor was required to fly from the start position to the goal position by following the global path while avoiding the dynamic obstacles.

E. DYNAMIC AVOIDANCE

The overall illustration of the flight test is shown in Fig.14-15. In the left image of Fig. 15, the black circles represent the position of the static obstacles, and the two diamonds roughly illustrate the direction of the motion of the dynamic obstacles.



FIGURE 14. Snapshots and depth detection in a human position during a physical flight test in an outdoor dynamic environment: (a).time stamp = 21 s, depth detection = 1.75 m. (b).time stamp = 37 s, depth detection = 2.14 m.

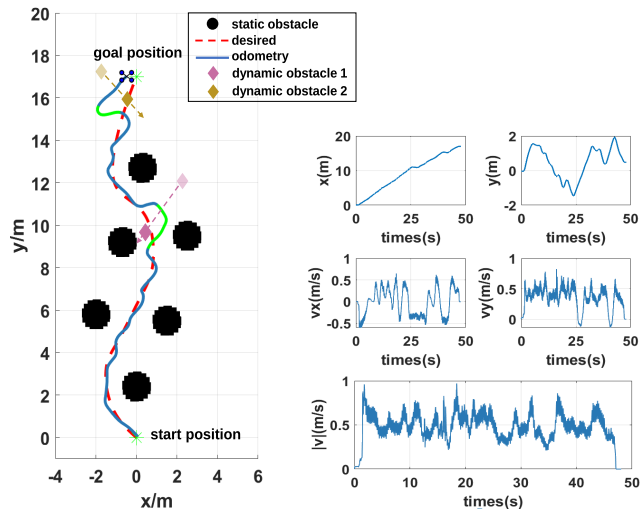


FIGURE 15. Left: illustration of a physical flight test in a dynamic environment. Right: position and velocity profile of the physical flight test based on onboard visual-initial odometry system.

The red-dashed line shows the generated global path, and the blue curve plots the odometry data from the onboard visual-initial estimation. The quadrotor began detouring at timestamps 21 s and 37 s to avoid colliding with people (see Fig.14). The avoidance part is marked in green. The right graph is the corresponding position and velocity plot. The total flight distance and flight time were 23 m and 48 s. The maximum velocity was set as 1 m/s, and the average flight velocity was about 0.48 m/s. The computational speed for the trajectory generation including uncertainty estimation is about 45ms in the computer used in Fig. 13. The optimization process is solved using OSQP(open-source-quadratic programming) solver [27].

VI. CONCLUSION

In this paper, we have presented a monocular camera-based dynamic avoidance framework for MAV outdoor navigation. The deep learning-based MDE algorithm extends the detection horizon in outdoor environments significantly. We also propose an uncertainty-aware trajectory prediction method to account for the uncertainties in a dynamic system. The proposed spatiotemporal trajectory planning method

succeeded in producing a safe and efficient avoidance trajectory. By using simulation and real-world experiments, we have verified that our work can be employed for steering MAVs away from dynamic obstacles. Our future work will focus on enhancing the robustness of the proposed method for a denser and more challenging environment.

REFERENCES

- [1] B. F. Jeon, Y. Lee, J. Choi, J. Park, and H. J. Kim, "Autonomous aerial dual-target following among obstacles," *IEEE Access*, vol. 9, pp. 143104–143120, 2021.
- [2] S. Lee and H. Lee, "Trajectory generation of a quadrotor transporting a bulky payload in the cluttered environments," *IEEE Access*, vol. 10, pp. 31586–31594, 2022.
- [3] T. Dang, M. Tranzatto, S. Khatkhat, F. Mascari, K. Alexis, and M. Hutter, "Graph-based subterranean exploration path planning using aerial and legged robots," *J. Field Robot.*, vol. 37, no. 8, pp. 1363–1388, Dec. 2020.
- [4] R. Mur-Artal and J. D. Tardós, "ORB-SLAM2: An open-source SLAM system for monocular, stereo, and RGB-D cameras," *IEEE Trans. Robot.*, vol. 33, no. 5, pp. 1255–1262, Oct. 2017.
- [5] T. Qin, P. Li, and S. Shen, "VINS-mono: A robust and versatile monocular visual-inertial state estimator," *IEEE Trans. Robot.*, vol. 34, no. 4, pp. 1004–1020, Aug. 2018.
- [6] Y. Wang, J. Ji, Q. Wang, C. Xu, and F. Gao, "Autonomous flights in dynamic environments with onboard vision," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, Sep. 2021, pp. 1966–1973.
- [7] Z. Xu, X. Zhan, B. Chen, Y. Xiu, C. Yang, and K. Shimada, "A real-time dynamic obstacle tracking and mapping system for UAV navigation and collision avoidance with an RGB-D camera," 2022, *arXiv:2209.08258*.
- [8] G. Chen, P. Peng, P. Zhang, and W. Dong, "Risk-aware trajectory sampling for quadrotor obstacle avoidance in dynamic environments," *IEEE Trans. Ind. Electron.*, vol. 70, no. 12, pp. 12606–12615, Dec. 2023.
- [9] N. J. Sanket, C. D. Singh, V. Asthana, C. Fermüller, and Y. Aloimonos, "MorphEyes: Variable baseline stereo for quadrotor navigation," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, May 2021, pp. 413–419.
- [10] F. Gao, W. Wu, Y. Lin, and S. Shen, "Online safe trajectory generation for quadrotors using fast marching method and Bernstein basis polynomial," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, May 2018, pp. 344–351.
- [11] B. Guo, N. Guo, and Z. Cen, "Obstacle avoidance with dynamic avoidance risk region for mobile robots in dynamic environments," *IEEE Robot. Autom. Lett.*, vol. 7, no. 3, pp. 5850–5857, Jul. 2022.
- [12] J. Lin, H. Zhu, and J. Alonso-Mora, "Robust vision-based obstacle avoidance for micro aerial vehicles in dynamic environments," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, May 2020, pp. 2682–2688.
- [13] A. B. Martinsen and A. M. Lekkas, "Two space-time obstacle representations based on ellipsoids and polytopes," *IEEE Access*, vol. 9, pp. 111152–111161, 2021.
- [14] J. Li, X. Xie, H. Ma, X. Liu, and J. He, "Speed planning using Bezier polynomials with trapezoidal corridors," 2021, *arXiv:2104.11655*.
- [15] A. Loquercio, A. I. Maqueda, C. R. Del-Blanco, and D. Scaramuzza, "DroNet: Learning to fly by driving," *IEEE Robot. Autom. Lett.*, vol. 3, no. 2, pp. 1088–1095, Apr. 2018.

- [16] P. Chakravarty, K. Kelchtermans, T. Roussel, S. Wellens, T. Tuytelaars, and L. Van Eycken, "CNN-based single image obstacle avoidance on a quadrotor," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, May 2017, pp. 6369–6374.
- [17] D. Eigen, C. Puhrsch, and R. Fergus, "Depth map prediction from a single image using a multi-scale deep network," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 27, 2014, pp. 1–9.
- [18] I. Alhashim and P. Wonka, "High quality monocular depth estimation via transfer learning," 2018, *arXiv:1812.11941*.
- [19] J. H. Lee, M.-K. Han, D. W. Ko, and I. H. Suh, "From big to small: Multi-scale local planar guidance for monocular depth estimation," 2019, *arXiv:1907.10326*.
- [20] S. F. Bhat, I. Alhashim, and P. Wonka, "AdaBins: Depth estimation using adaptive bins," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2021, pp. 4008–4017.
- [21] T. Van Dijk and G. De Croon, "How do neural networks see depth in single images?" in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2019, pp. 2183–2191.
- [22] *YoloV5*. Accessed: 2023. [Online]. Available: <https://github.com/ultralytics/yolov5>
- [23] R. Ranftl, K. Lasinger, D. Hafner, K. Schindler, and V. Koltun, "Towards robust monocular depth estimation: Mixing datasets for zero-shot cross-dataset transfer," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 44, no. 3, pp. 1623–1637, Mar. 2022.
- [24] Z. Han, R. Zhang, N. Pan, C. Xu, and F. Gao, "Fast-tracker: A robust aerial system for tracking agile target in cluttered environments," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, May 2021, pp. 328–334.
- [25] F. Furrer, M. Burri, M. Achtelik, and R. Siegwart, "RotorS—A modular gazebo MAV simulator framework," in *Robot Operating System (ROS): The Complete Reference*, vol. 1. Cham, Switzerland: Springer, 2016, pp. 595–625.
- [26] X. Zhou, Z. Wang, H. Ye, C. Xu, and F. Gao, "EGO-planner: An ESDF-free gradient-based local planner for quadrotors," *IEEE Robot. Autom. Lett.*, vol. 6, no. 2, pp. 478–485, Apr. 2021.
- [27] B. Stellato, G. Banjac, P. Goulart, A. Bemporad, and S. Boyd, "OSQP: An operator splitting solver for quadratic programs," *Math. Program. Comput.*, vol. 12, no. 4, pp. 637–672, Dec. 2020.



MIAOJUN ZHOU received the B.S. degree from the Department of Mechanical Engineering, Busan National University, in 2019. He is currently pursuing the M.S. degree with the School of Electronic and Electrical Engineering, Kyungpook National University, Daegu, South Korea. His current research interests include mobile robot navigation and SLAM.



HYEONBEOM LEE (Member, IEEE) received the B.S. degree in mechanical and control engineering from Handong Global University, in 2011, and the M.S. and Ph.D. degrees in mechanical and aerospace engineering from Seoul National University, in 2013 and 2017, respectively. From 2017 to 2018, he was a Senior Researcher with Korea Institute of Machinery and Materials (KIMM). From 2018 to 2023, he was an Assistant Professor Researcher with Kyungpook National University. In September 2023, he joined the Department of Mechanical Engineering, Ajou University, Suwon, South Korea, as an Assistant Professor. His research interest includes the autonomous navigation of mobile robots.

...